

# **Clase teórica 9**

## **Lógica de Hoare**

# Componentes de la Lógica de Hoare

## 1. Lenguaje de programación

- Instrucciones:

$S :: \text{skip} \mid x := e \mid S_1 ; S_2 \mid \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi} \mid \text{while } B \text{ do } S_1 \text{ od}$

Agregamos la instrucción `skip`, que no modifica el estado inicial (útil p.ej. para simular un `if` sin `else`)

- Expresiones de tipo entero:

$e :: n \mid x \mid e_1 + e_2 \mid e_1 - e_2 \mid e_1 \cdot e_2 \mid \dots$

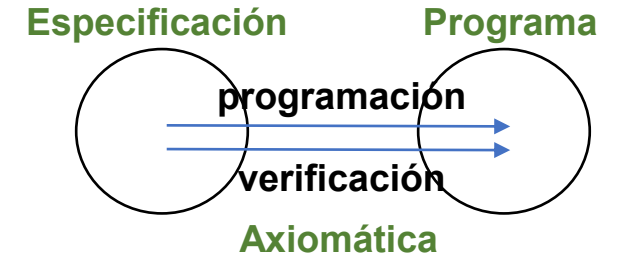
$n$  es una constante entera.  $x$  es una variable entera.

- Expresiones de tipo booleano:

$B :: \text{true} \mid \text{false} \mid e_1 = e_2 \mid e_1 < e_2 \mid \dots \mid \neg B_1 \mid B_1 \vee B_2 \mid B_1 \wedge B_2 \mid \dots$

## 2. Lenguaje de especificación (lógica de predicados):

$p :: \text{true} \mid \text{false} \mid e_1 = e_2 \mid e_1 < e_2 \mid \dots \mid \neg p \mid p_1 \vee p_2 \mid \dots \mid \exists x: p \mid \forall x: p$



### Ejemplo de programa

```
a := 1 ; y := 1 ;  
while a < x do  
  a := a + 1 ; y := y . a  
od
```

### Ejemplos de predicados

```
true  
x + 1 = y  
 $\neg(a < x)$   
 $\forall x: (x > y \vee x \leq y)$ 
```

## 3.1. Axiomática para correctitud parcial (prueba parcial de los while) MÉTODO H

### 1. Axioma del skip (SKIP)

$$\{p\} \text{ skip } \{p\}$$

### 2. Axioma de la asignación (ASI)

$$\{p[x|e]\} x := e \{p\}$$

$p[x|e]$  denota la sustitución en el predicado  $p$  de toda ocurrencia (libre) de la variable entera  $x$  por la expresión entera  $e$

### 3. Regla de la secuencia (SEC)

$$\frac{\{p\} S_1 \{r\}, \{r\} S_2 \{q\}}{\{p\} S_1 ; S_2 \{q\}}$$

### 4. Regla del condicional (COND)

$$\frac{\{p \wedge B\} S_1 \{q\}, \{p \wedge \neg B\} S_2 \{q\}}{\{p\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \text{ fi } \{q\}}$$

### 5. Regla de la repetición (REP)

No asegura que el while termina

$$\frac{\{p \wedge B\} S \{p\}}{\{p\} \text{ while } B \text{ do } S \text{ od } \{p \wedge \neg B\}}$$

El predicado  $p$  es un predicado invariante del while

### 6. Regla de consecuencia (CONS)

$$\frac{p \rightarrow p_1, \{p_1\} S \{q_1\}, q_1 \rightarrow q}{\{p\} S \{q\}}$$

Se puede reemplazar  $p_1$  por un  $p$  que lo implique, y/o  $q_1$  por un  $q$  al que implique

- Llama la atención la forma del axioma de la asignación (ASI).

El axioma establece  $\{p[x|e]\} x := e \{p\}$ .

Se lee así: si luego de  $x := e$  vale  $p$  en términos de  $x$ , entonces antes de  $x := e$  valía  $p$  en términos de  $e$ .

Por ejemplo, veamos cómo se completa la fórmula  $\{?\} x := x + 1 \{x \geq 0\}$ :

$\{x \geq 0[x|x + 1]\} x := x + 1 \{x \geq 0\}$

$\{x + 1 \geq 0\} x := x + 1 \{x \geq 0\}$

Es decir, si luego de  $x := x + 1$  vale  $x \geq 0$ , entonces antes de  $x := x + 1$  valía  $x + 1 \geq 0$ .

El axioma se lee de atrás para adelante, impone una forma de probar de la postcondición a la precondición.

Sería más natural la forma de axioma hacia adelante:  $\{\text{true}\} x := e \{x = e\}$ ,  
donde **true** es un predicado que representa cualquier condición.

Pero esta fórmula de correctitud no siempre es verdadera. Por ejemplo:

$\{\text{true}\} x := x + 1 \{x = x + 1\}$  es una fórmula falsa.

El contenido de la  $x$  de la derecha es previo a la asignación, y el de la  $x$  de la izquierda es posterior.

Existe un axioma correcto hacia adelante, más complicado y no suele usarse:  $\{p\} x := e \{\exists z: (p[x|z] \wedge x = e[x|z])\}$ .

- En la **regla de la secuencia (SEC)**, el predicado **r** actúa como **nexo** y luego **se descarta**, no se propaga.

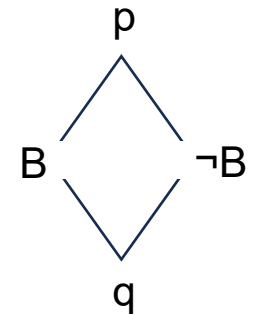
$$\frac{\{p\} S_1 \{r\}, \{r\} S_2 \{q\}}{\{p\} S_1 ; S_2 \{q\}}$$

Se permite usar la siguiente **generalización**:

$$\frac{\{p\} S_1 \{r_1\}, \{r_1\} S_2 \{r_2\}, \dots, \{r_{n-1}\} S_n \{q\}}{\{p\} S_1 ; S_2 ; \dots ; S_n \{q\}}$$

- La **regla del condicional (COND)** formula un modo de verificar una selección condicional fijando un **único punto de entrada** y un **único punto de salida**, correspondientes a **p** y **q**, respectivamente.

$$\frac{\{p \wedge B\} S_1 \{q\}, \{p \wedge \neg B\} S_2 \{q\}}{\{p\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \text{ fi } \{q\}}$$

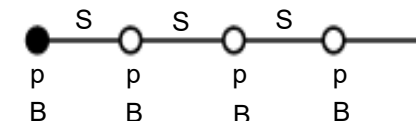


- La **regla de la repetición (REP)** se basa en un **invariante p**. Si **p** vale al comienzo del *while*, y mientras vale **B** el cuerpo **S** preserva **p**, entonces por un razonamiento **inductivo** **p** vale al finalizar el *while*.

$$\frac{\{p \wedge B\} S \{p\}}{\{p\} \text{ while } B \text{ do } S \text{ od } \{p \wedge \neg B\}}$$

Esta inducción se conoce como *inducción computacional*.

Claramente, REP no asegura que el *while* termina.



- La **regla de consecuencia (CONS)** permite **reforzar precondiciones** y **debilitar postcondiciones**. P.ej.:

de:  $\{x > 0\} S \{x = 0\}$   
 y:  $x > 5 \rightarrow x > 0$   
 se deduce:  $\{x > 5\} S \{x = 0\}$

de:  $\{\text{true}\} S \{x = y + 1\}$   
 y:  $x = y + 1 \rightarrow x > y$   
 se deduce:  $\{\text{true}\} S \{x > y\}$

$$\frac{p \rightarrow p_1, \{p_1\} S \{q_1\}, q_1 \rightarrow q}{\{p\} S \{q\}}$$

La regla no depende del lenguaje de programación sino del dominio semántico.

Es una regla **semántica** más que sintáctica.

Permite utilizar **todos los axiomas del dominio semántico** (en este caso los enteros).

**Nota: así, la incompletitud de la aritmética “condena” al método, usando enteros, a ser incompleto.**  
 (de todos modos, de lo que se trata es de probar programas, no enunciados de la aritmética)

- El método es **composicional**:

Dado un programa  $S$  con subprogramas  $S_1, \dots, S_n$ , que valga la fórmula  $\{p\} S \{q\}$  depende sólo de que valgan ciertas fórmulas  $\{p_1\} S_1 \{q_1\}, \dots, \{p_n\} S_n \{q_n\}$ , sin importar el contenido de los  $S_i$  (noción de **caja negra**).

P.ej., dado  $S :: S_1 ; S_2$ ,  
 de  $\{p\} S_1 \{r\}$  y  $\{r\} S_2 \{q\}$   
 se deduce  $\{p\} S_1 ; S_2 \{q\}$ ,  
 independientemente de los contenidos de  $S_1$  y  $S_2$ .

**Si se tiene  $\{r\} S_3 \{q\}$  también se deduce  $\{p\} S_1 ; S_3 \{q\}$ .  
 $S_2$  y  $S_3$  son intercambiables por ser funcionalmente equivalentes en relación a  $(r, q)$ .**

## Ejemplo 1. Prueba de un programa que intercambia los valores de dos variables

- Dado  $S_{\text{swap}} :: z := x ; x := y ; y := z$ , se va a probar:

$$\{x = X \wedge y = Y\} S_{\text{swap}} \{y = X \wedge x = Y\}$$

Antes lo probamos **semánticamente**, usando la semántica del lenguaje de programación.

Ahora lo probaremos **sintácticamente**, usando axiomas y reglas (antes hemos mostrado la idea general).

Por la forma del programa  $S_{\text{swap}}$  recurrimos al axioma ASI tres veces, y al final completamos con la (generalización de la) regla SEC:

$$1. \{z = X \wedge x = Y\} y := z \{y = X \wedge x = Y\} \quad (\text{ASI})$$

$$2. \{z = X \wedge y = Y\} x := y \{z = X \wedge x = Y\} \quad (\text{ASI})$$

$$3. \{x = X \wedge y = Y\} z := x \{z = X \wedge y = Y\} \quad (\text{ASI})$$

$$4. \{x = X \wedge y = Y\} z := x ; x := y ; y := z \{y = X \wedge x = Y\} \quad (1, 2, 3, \text{SEC})$$

**Axioma ASI**  
 $\{p[x|e]\} x := e \{p\}$

**Regla SEC**  
 $\frac{\{p\} S_1 \{r_1\}, \dots, \{r_{n-1}\} S_n \{q\}}{\{p\} S_1 ; \dots ; S_n \{q\}}$

- Notar cómo el axioma ASI establece una forma de prueba **de la postcondición a la precondition**.
- Por la **sensatez** del método (que vamos a probar la clase que viene), haber probado **sintácticamente**:

$$\{x = X \wedge y = Y\} z := x ; x := y ; y := z \{y = X \wedge x = Y\}$$

significa que la fórmula se cumple **semánticamente**.

- Obviamente, también se cumple **semánticamente** la precondition con los operandos permutados:

$$\{y = Y \wedge x = X\} z := x ; x := y ; y := z \{y = X \wedge x = Y\}$$

- Para probar esta fórmula **sintácticamente** recurrimos a la **regla de consecuencia (CONS)**:

Agregamos a la prueba anterior, que terminaba con:

$$4. \{x = X \wedge y = Y\} z := x ; x := y ; y := z \{y = X \wedge x = Y\} \quad (1, 2, 3, \text{SEC})$$

$$\frac{\text{Regla CONS} \quad p \rightarrow p_1, \{p_1\} S \{q_1\}, q_1 \rightarrow q}{\{p\} S \{q\}}$$

los siguientes dos pasos:

$$5. (y = Y \wedge x = X) \rightarrow (x = X \wedge y = Y) \quad (\text{MAT})$$

$$6. \{y = Y \wedge x = X\} z := x ; x := y ; y := z \{y = X \wedge x = Y\} \quad (4, 5, \text{CONS})$$



- El predicado del paso 5 es un axioma de los enteros, por eso usamos el indicador **MAT** (por matemáticas).
- Así, hemos probado la fórmula de correctitud:

$$\{y = Y \wedge x = X\} S_{\text{swap}} \{y = X \wedge x = Y\}$$

- Y si quisiéramos **instanciarla**, por ejemplo en:

$$\{y = 2 \wedge x = 1\} S_{\text{swap}} \{y = 1 \wedge x = 2\}$$

debemos recurrir a una nueva regla, la **regla de instanciación (INST)**:

$$\frac{f(X)}{f(c)}$$

tal que  $f$  es una fórmula de correctitud,  $X$  es una variable lógica, y  $c$  está en el dominio de  $X$ .

INST es una regla universal, se la usa en todos los métodos, a pesar de que no se la suele mencionar.

## Ejemplo 2. Prueba de un programa que calcula el valor absoluto

- El siguiente programa calcula en una variable y el valor absoluto de una variable x:

$S_{va} :: \text{if } x > 0 \text{ then } y := x \text{ else } y := -x \text{ fi}$

**Axioma ASI**  
 $\{p[x|e]\} x := e \{p\}$

**Vamos a probar  $\{true\} S_{va} \{y \geq 0\}$**

**Regla COND**  
 $\frac{\{p \wedge B\} S_1 \{q\}, \{p \wedge \neg B\} S_2 \{q\}}{\{p\} \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi } \{q\}}$

- Por el if then else, hay que usar la regla COND.

Para llegar a:  $\{true\} \text{if } x > 0 \text{ then } y := x \text{ else } y := -x \text{ fi } \{y \geq 0\}$

tenemos que obtener:  $\{true \wedge x > 0\} y := x \{y \geq 0\}$ ,  $\{true \wedge \neg(x > 0)\} y := -x \{y \geq 0\}$

- Considerando las dos asignaciones del programa, los primeros pasos de la prueba podrían ser:

1.  $\{x \geq 0\} y := x \{y \geq 0\}$  (ASI)

2.  $\{-x \geq 0\} y := -x \{y \geq 0\}$  (ASI)

- Veamos cómo podemos combinar el axioma ASI con la regla COND para lograr la prueba.

Comenzamos por las asignaciones:

$$1. \{x \geq 0\} y := x \{y \geq 0\} \quad (\text{ASI})$$

$$2. \{-x \geq 0\} y := -x \{y \geq 0\} \quad (\text{ASI})$$

Luego hacemos:

$$3. (\text{true} \wedge x > 0) \rightarrow x \geq 0 \quad (\text{MAT})$$

$$4. \{\text{true} \wedge x > 0\} y := x \{y \geq 0\} \quad (1, 3, \text{CONS})$$

$$5. (\text{true} \wedge \neg(x > 0)) \rightarrow -x \geq 0 \quad (\text{MAT})$$

$$6. \{\text{true} \wedge \neg(x > 0)\} y := -x \{y \geq 0\} \quad (2, 5, \text{CONS})$$

Finalmente:

$$7. \{\text{true}\} \text{ if } x > 0 \text{ then } y := x \text{ else } y := -x \text{ fi } \{y \geq 0\} \quad (4, 6, \text{COND})$$

$S_{va} :: \text{ if } x > 0 \text{ then } y := x \text{ else } y := -x \text{ fi}$

**Axioma ASI**  
 $\{p[x|e]\} x := e \{p\}$

**Regla CONS**  
$$\frac{p \rightarrow p_1, \{p_1\} S \{q_1\}, q_1 \rightarrow q}{\{p\} S \{q\}}$$

**Regla COND**  
$$\frac{\{p \wedge B\} S_1 \{q\}, \{p \wedge \neg B\} S_2 \{q\}}{\{p\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \text{ fi } \{q\}}$$

### Ejemplo 3. Prueba de correctitud parcial de un programa que calcula el factorial

- Se verá en la clase práctica:

$$\begin{array}{l} \{x > 0\} \\ S_{\text{fac}} :: a := 1 ; y := 1 ; \\ \quad \text{while } a < x \text{ do} \\ \quad \quad a := a + 1 ; y := y \cdot a \\ \quad \text{od} \\ \{y = x!\} \end{array}$$

**Dado  $x > 0$ , si  $S_{\text{fac}}$  termina obtiene  $y = x!$**

- Un **invariante** del while posible es:  $p = (y = a! \wedge a \leq x)$

La idea es que se vaya calculando en la variable  $y$  el factorial de  $a$ , hasta que  $a = x$ .

Notar que el invariante es similar a la postcondición: cuando  $a = x$ , entonces  $p = (y = x!)$

- La prueba se puede estructurar de la siguiente manera:

a)  $\{x > 0\} a := 1 ; y := 1 \{y = a! \wedge a \leq x\}$

Se cumple el invariante por primera vez

b)  $\{y = a! \wedge a \leq x\}$

$\text{while } a < x \text{ do } a := a + 1 ; y := y \cdot a \text{ od}$   
 $\{(y = a! \wedge a \leq x) \wedge \neg(a < x)\} \equiv \{y = x!\}$

Aplicación de REP

c)  $\{x > 0\} S_{\text{fac}} \{y = x!\}$

Aplicación de SEC sobre (a) y (b)

#### Regla REP

$$\frac{\{p \wedge B\} S \{p\}}{\{p\} \text{ while } B \text{ do } S \text{ od } \{p \wedge \neg B\}}$$

#### Regla SEC

$$\frac{\{p\} S_1 \{r_1\}, \dots, \{r_{n-1}\} S_n \{q\}}{\{p\} S_1 ; \dots ; S_n \{q\}}$$

## 3.2. Axiomática para la correctitud total (prueba total de los while) MÉTODO H\*

- La regla REP sólo permite probar la **invariancia de un predicado p**.
- La **regla REP\*** permite probar además la **terminación del while**.
- La regla REP\* se basa en un **predicado invariante p** y una **función variante t**.

t es una función entera definida, como el invariante, en términos de las variables de programa.

$$\text{Regla REP*}: \frac{\langle p \wedge B \rangle S \langle p \rangle, \quad \langle p \wedge B \wedge (t = Z) \rangle S \langle t < Z \rangle, \quad p \rightarrow t \geq 0}{\langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle}$$

Se agregan dos premisas a REP, y se reemplazan los símbolos { } por < >.

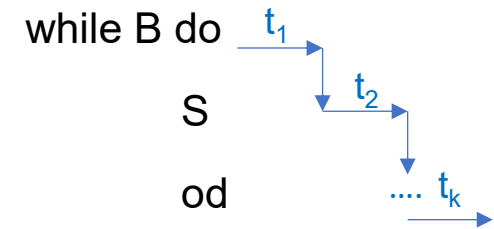
Por la segunda premisa, **t se decrementa en cada iteración**.

Z es una variable lógica, no aparece en p, B, t ni S.

El objetivo de Z es fijar el valor de t antes de la ejecución de S.

Por la tercera premisa, **t se mantiene  $\geq 0$  a lo largo de todo el while**.

Así, el while debe terminar, porque en  $\mathcal{N}$  (números naturales) **no hay cadenas descendentes infinitas**



La función t representa el máximo número de iteraciones del while

**$(\mathcal{N}, <)$  es un ejemplo de relación de orden bien fundada (todo subconjunto tiene minimal).**

## Ejemplo 4. Prueba de terminación del programa del factorial

### Regla REP\*

$$\frac{\langle p \wedge B \rangle S \langle p \rangle , \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle , p \rightarrow t \geq 0}{\langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle}$$

- Se verá en la clase práctica.

- Con el método H se puede probar:

$$\{x > 0\} S_{\text{fac}} :: a := 1 ; y := 1 ; \text{ while } a < x \text{ do } a := a + 1 ; y := y \cdot a \text{ od } \{y = x!\}$$

utilizando p.ej. el invariante:  $p = (y = a! \wedge a \leq x)$

**Si  $S_{\text{fac}}$  termina lo hace con la postcondición  $y = x!$**

- Con el método H\* se puede probar:

$$\langle x > 0 \rangle S_{\text{fac}} :: a := 1 ; y := 1 ; \text{ while } a < x \text{ do } a := a + 1 ; y := y \cdot a \text{ od } \langle \text{true} \rangle$$

**$S_{\text{fac}}$  termina (con cualquier postcondición, es irrelevante).**

- Juntando las dos pruebas:  $\langle x > 0 \rangle S_{\text{fac}} :: a := 1 ; y := 1 ; \text{ while } a < x \text{ do } a := a + 1 ; y := y \cdot a \text{ od } \langle y = x! \rangle$

- Para la prueba de terminación del while podrían utilizarse:

Invariante  $p = (a \leq x)$       **Notar que este invariante es más simple que el anterior.**

Variante  $t = x - a$       Al comienzo,  $t \geq 0$ . A lo largo de las iteraciones  $t$  se decrementa en 1. Al final,  $t = 0$ .

- ¿Por qué separar la prueba del while en dos, correctitud parcial y terminación?
- Porque son dos pruebas distintas, utilizan técnicas distintas:
  - Correctitud parcial: **inducción** (invariante que vale al comienzo y luego de cada iteración).
  - Terminación: **relación de orden bien fundada** (variante entero  $\geq 0$  que se decrementa tras cada iteración).
- En verdad, la regla REP\* permite probar todo junto, pero la recomendación es probar  $\langle p \rangle S \langle q \rangle$  en dos pasos:

(a)  $\{p\} S \{q\}$   
 (b)  $\langle p \rangle S \langle \text{true} \rangle$

- (a) Si S termina a partir de p, lo hace en q.  
 (b) S termina a partir de p.  
 (a, b) S termina a partir de p en q.

|                                                                                                                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\frac{\langle p \wedge B \rangle S \langle p \rangle, \quad \langle p \wedge B \wedge (t = Z) \rangle S \langle t < Z \rangle, \quad p \rightarrow t \geq 0}{\langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle q \rangle, \text{ tal que } (p \wedge \neg B) \rightarrow q}$ |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- Por ejemplo:

$\langle x > 0 \rangle S_{\text{fac}} :: a := 1 ; y := 1 ; \text{ while } a < x \text{ do } a := a + 1 ; y := y \cdot a \text{ od } \langle y = x! \rangle$

Para probar  $\{x > 0\} S_{\text{fac}} \{y = x!\}$  se puede utilizar el invariante  **$y = a! \wedge a \leq x$** ,

y para probar  $\langle x > 0 \rangle S_{\text{fac}} \langle \text{true} \rangle$ , un invariante más simple:  **$a \leq x$** . **evitando arrastrar información innecesaria.**

**Anexo**



## Proof outlines (esquemas de prueba)

- Presentación alternativa de una prueba: se intercalan los pasos de la prueba entre las instrucciones del programa.
- Se obtiene una prueba más estructurada, que documenta adecuadamente el programa.
- En la verificación de los programas concurrentes las *proof outlines* son **imprescindibles**.
- Por ejemplo, la siguiente es una *proof outline* de correctitud parcial de un programa que calcula el factorial:

```
{x > 0}
Sfac :: a := 1 ; y := 1 ;
      while a < x do
        a := a + 1 ; y := y . a
      od
{y = x!}
```

```
{x > 0}
a := 1 ;
y := 1 ;
{inv: y = a! ∧ a ≤ x}
while a < x do
  {y = a! ∧ a < x}
  a := a + 1 ;
  y := y . a
od
{(y = a! ∧ a ≤ x) ∧ ¬(a < x)}
{y = x!}
```

En una proof outline de correctitud total se agrega el variante.

- Mínimamente se suelen documentar la **precondición**, la **postcondición**, los **invariantes** y los **variantes**.

# Invariantes, variantes, y propiedades *safety* (seguridad) y *liveness* (progreso)

- Todo predicado utilizado en una prueba es en realidad un **invariante**. Volviendo a las *proof outlines*: cualquiera sea el estado inicial, todo predicado siempre se cumple en el lugar donde se establece:

```
{x > 0}
a := 1 ; y := 1 ;
{y = a! ∧ a ≤ x}
while a < x do
  {y = a! ∧ a < x}
  a := a + 1 ; y := y . a
od
{y = x!}
```

- El uso de un invariante para la prueba de un *while* determina una prueba por **inducción**: es un predicado que vale al comienzo del *while* (base inductiva) y que toda iteración preserva (paso inductivo). De esta manera se prueba que el invariante se cumple **a lo largo de toda la computación** del *while*.
- Ligado a lo anterior, la correctitud parcial pertenece a la familia de las propiedades **safety**. Son propiedades que se prueban por **inducción**. Se asocian al enunciado: “Algo malo no puede suceder”. Otros ejemplos son la *ausencia de deadlock* y la *exclusión mutua* o *ausencia de interferencia*, en los programas concurrentes.
- La terminación o no divergencia, en cambio, pertenece a la familia de las propiedades **liveness**, basadas en **funciones variantes** definidas en **relaciones de orden bien fundadas**. Se asocian al enunciado: “Algo bueno va a suceder”. Otro ejemplo es la *ausencia de inanición* (*non starvation*), en los programas concurrentes.

## Axiomas y reglas adicionales

- Por la **completitud** del método de prueba estudiado (que se prueba en la clase que viene), agregarle axiomas y reglas resulta **redundante**. De todos modos, esta práctica es usual en los sistemas deductivos, facilita y acorta las pruebas. Algunos ejemplos clásicos de axiomas y reglas adicionales son:

- Axioma de invariancia (INV)  $\{p\} S \{p\}$   
Cuando las variables libres de  $p$  y las variables que modifica  $S$  **son disjuntas**.

- Regla de la disyunción (OR)
$$\frac{\{p\} S \{q\}, \{r\} S \{q\}}{\{p \vee r\} S \{q\}}$$

Util para una **verificación por casos**.

- Regla de la conjunción (AND)
$$\frac{\{p_1\} S \{q_1\}, \{p_2\} S \{q_2\}}{\{p_1 \wedge p_2\} S \{q_1 \wedge q_2\}}$$

También útil para una **verificación por casos**.

- El axioma INV suele emplearse en combinación con la regla AND, para producir la **Regla de invariancia (RINV)**:

$$\frac{\{p\} S \{q\}}{\{r \wedge p\} S \{r \wedge q\}}$$

tal que **ninguna variable libre de r es modificable por S** (se cumple  $\{r\} S \{r\}$ ).

- Dada la Regla de la Disyunción (OR):

$$\frac{\{p\} S \{q\} , \{r\} S \{q\}}{\{p \vee r\} S \{q\}}$$

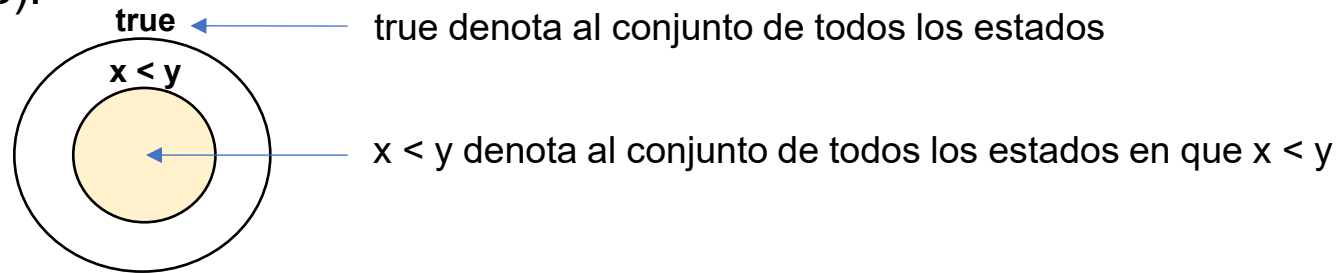
una forma particular de la regla de uso habitual es:

$$\frac{\{p \wedge s\} S \{q\} , \{p \wedge \neg s\} S \{q\}}{\{p\} S \{q\}}$$

útil cuando la prueba de  $\{p\} S \{q\}$  se facilita reforzando la precondition con **dos aserciones complementarias**.

# Sintaxis y semántica. Abstracción.

- Un **predicado** (elemento sintáctico) representa un **conjunto de estados** (elemento semántico), el conjunto de todos los estados que satisfacen el predicado. P.ej.,  $x < y$  denota a todos los estados tales que  $x < y$ . En particular, **true** denota a todos los estados y **false** denota al conjunto vacío de estados (para todo estado  $\sigma$ , se cumple  $\sigma \models \text{true}$  y  $\sigma \not\models \text{false}$ ).

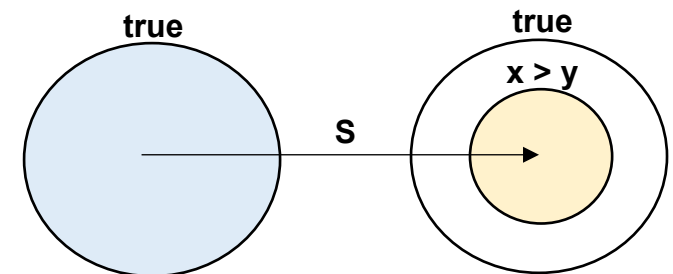


- Una **especificación** de un programa  $S$  es un par de predicados  $(p, q)$  asociados a la **entrada** y la **salida** de  $S$ , respectivamente,  $p$  denota al conjunto de estados iniciales de  $S$ , y  $q$  al conjunto de estados finales de  $S$ .
- Por ejemplo,  $(x = X, x = 2X)$  es satisfecha por un programa que duplica su entrada  $x$  (como  $S :: x := x + x$ ). La variable  $x$  es una **variable de programa**. La variable  $X$  es una **variable lógica o de especificación** (no es parte del programa, se usa para fijar valores).

**Ejercicio. Especificar un programa  $S$  que termine con la postcondición  $x > y$ .**

Una posible especificación podría ser:  $(x = X \wedge y = Y, x > y)$

Otra más simple podría ser:  $(\text{true}, x > y)$



A partir de la precondition true, el programa  $S$  debe terminar en la postcondición  $x > y$

# Correctitud parcial y total de un programa

- $\pi(S, \sigma)$  denota la computación de un programa  $S$  a partir de un estado inicial  $\sigma$ .
- $\text{val}(\pi(S, \sigma)) = \sigma'$  denota el estado final de  $\pi(S, \sigma)$ .  
En particular,  $\text{val}(\pi(S, \sigma)) = \perp$  denota que  $\pi(S, \sigma)$  no termina.
- Un programa  $S$  es **correcto parcialmente** con respecto a una especificación  $(p, q)$  sii:  
Para todo estado  $\sigma$ :  $[\sigma \models p \wedge \text{val}(\pi(S, \sigma)) = \sigma' \neq \perp] \rightarrow \sigma' \models q$   
es decir, a partir de un estado  $\sigma \models p$ , **si  $S$  termina (o no diverge)** lo hace en un estado  $\sigma' \models q$ .
- Un programa  $S$  es **correcto totalmente** con respecto a una especificación  $(p, q)$  sii:  
Para todo estado  $\sigma$ :  $\sigma \models p \rightarrow [\text{val}(\pi(S, \sigma)) = \sigma' \neq \perp \wedge \sigma' \models q]$   
es decir, a partir de un estado  $\sigma \models p$ ,  **$S$  termina (o no diverge)** lo hace en un estado  $\sigma' \models q$ .
- $\{p\} S \{q\}$  denota la correctitud parcial y  $\langle p \rangle S \langle q \rangle$  denota la correctitud total.

Las dos propiedades se distinguen porque se prueban con **métodos distintos**.

Ejercicio (asumiendo que *true* representa cualquier estado)

1. ¿Se cumple  $\{x = 11\} \text{ while } x \neq 0 \text{ do } x := x - 2 \text{ od } \{true\}$ ?
2. ¿Se cumple  $\langle x = 11 \rangle \text{ while } x \neq 0 \text{ do } x := x - 2 \text{ od } \langle true \rangle$ ?

# Lema de Separación

**Prueba formal de que  $\langle p \rangle S \langle q \rangle \leftrightarrow (\{p\} S \{q\} \wedge \langle p \rangle S \langle \text{true} \rangle)$**

- Primero probamos  $\langle p \rangle S \langle q \rangle \rightarrow (\{p\} S \{q\} \wedge \langle p \rangle S \langle \text{true} \rangle)$ :

Sea  $\langle p \rangle S \langle q \rangle$ . Entonces, dado  $\sigma$ , vale  $\sigma \models p \rightarrow (\text{val}(\pi(S, \sigma)) \neq \perp \wedge \text{val}(\pi(S, \sigma)) \models q)$ . Por lo tanto:

(a)  $(\sigma \models p \wedge \text{val}(\pi(S, \sigma)) \neq \perp) \rightarrow \text{val}(\pi(S, \sigma)) \models q$ , es decir  $\{p\} S \{q\}$ .

(b)  $\sigma \models p \rightarrow (\text{val}(\pi(S, \sigma)) \neq \perp \wedge \text{val}(\pi(S, \sigma)) \models \text{true})$ , es decir  $\langle p \rangle S \langle \text{true} \rangle$ .

Así, por (a) y (b):  $\{p\} S \{q\} \wedge \langle p \rangle S \langle \text{true} \rangle$ .

- Ahora probamos  $(\{p\} S \{q\} \wedge \langle p \rangle S \langle \text{true} \rangle) \rightarrow \models \langle p \rangle S \langle q \rangle$ :

Sea  $\{p\} S \{q\} \wedge \langle p \rangle S \langle \text{true} \rangle$ . Entonces, dado  $\sigma$ , vale:

(a)  $(\sigma \models p \wedge \text{val}(\pi(S, \sigma)) \neq \perp) \rightarrow \text{val}(\pi(S, \sigma)) \models q$ .

(b)  $\sigma \models p \rightarrow (\text{val}(\pi(S, \sigma)) \neq \perp \wedge \text{val}(\pi(S, \sigma)) \models \text{true})$ .

(c) Por (b):  $\sigma \models p \rightarrow \text{val}(\pi(S, \sigma)) \neq \perp$ .

(d) Por (a) y (b):  $\sigma \models p \rightarrow \text{val}(\pi(S, \sigma)) \models q$ .

Así, por (c) y (d):  $\sigma \models p \rightarrow (\text{val}(\pi(S, \sigma)) \neq \perp \wedge \text{val}(\pi(S, \sigma)) \models q)$ , es decir  $\langle p \rangle S \langle q \rangle$ .

## Sensatez total

- Se probará en la práctica:  $\{x \geq 0 \wedge y > 0\} S_{\text{div}} \{x = y \cdot c + r \wedge r < y \wedge r \geq 0\}$

siendo:  $S_{\text{div}} :: c := 0 ; r := x ; \text{while } r \geq y \text{ do } r := r - y ; c := c + 1 \text{ od}$

y la estructura general de la prueba (p es el invariante):

- a)  $\{x \geq 0 \wedge y > 0\} c := 0 ; r := x \{p\}$
- b)  $\{p\} \text{while } r \geq y \text{ do } r := r - y ; c := c + 1 \text{ od } \{p \wedge \neg(r \geq y)\}$
- c)  $\{x \geq 0 \wedge y > 0\} S_{\text{div}} \{x = y \cdot c + r \wedge r < y \wedge r \geq 0\}$

La misma prueba sirve para probar el programa con variables reales.

## Completitud aritmética

- Sea el siguiente programa, con variables x y  $\varepsilon$  de tipo real inicialmente mayores que 0:

$S :: \text{while } x > \varepsilon$   
    do  
     $x := x / 2$   
    od

La función t tiene que ser entera, independientemente del tipo de las variables de programa.

Se cumple:  $\langle x = X \wedge X > 0 \wedge \varepsilon > 0 \rangle S \langle \text{true} \rangle$

y una posible función entera t para la prueba de terminación es:  $t = \text{if } x > \varepsilon \text{ then } \lceil \log_2 X/\varepsilon \rceil - \log_2 X/x \text{ else } 0 \text{ fi}$



# **Clase práctica 11**

## Ejemplo 1. Prueba de correctitud parcial de un programa que calcula el factorial

- Vamos a probar con el método H:

$$S_{\text{fac}} :: \begin{array}{l} \{x > 0\} \\ a := 1 ; y := 1 ; \text{while } a < x \text{ do } a := a + 1 ; y := y \cdot a \text{ od} \\ \{y = x!\} \end{array}$$

Es decir, dado  $x > 0$ , el programa  $S_{\text{fac}}$ , si termina, obtiene  $y = x!$

**Ejercicio:** ¿Es correcta la especificación  $(x > 0, y = x!)$  para un programa que calcule el factorial?

- Se propone como **invariante** del while:

$$p = (y = a! \wedge a \leq x)$$

La idea es que se vaya calculando en  $y$  el factorial de  $a$ , hasta que  $a = x$ . En efecto, notar que el invariante propuesto es similar a la postcondición, **la generaliza**, sustituyendo  $x$  por  $a$ . Cuando  $a = x$ , entonces  $y = x!$ .

- La prueba se puede estructurar de la siguiente manera:

a)  $\{x > 0\} a := 1 ; y := 1 \{y = a! \wedge a \leq x\}$

Se cumple el invariante por primera vez

b)  $\{y = a! \wedge a \leq x\}$

$\text{while } a < x \text{ do } a := a + 1 ; y := y \cdot a \text{ od}$   
 $\{y = x!\}$

Aplicación de REP y CONS

c)  $\{x > 0\} S_{\text{fac}} \{y = x!\}$

Aplicación de SEC sobre (a) y (b)

$$\frac{\{p \wedge B\} S \{p\}}{\{p\} \text{ while } B \text{ do } S \text{ od } \{p \wedge \neg B\}}$$

$$\{p\} \text{ while } B \text{ do } S \text{ od } \{p \wedge \neg B\}$$

**Recordatorio de la regla REP**

## Prueba de (a)

1.  $\{1 = a! \wedge a \leq x\} y := 1 \{y = a! \wedge a \leq x\}$  (ASI)
2.  $\{1 = 1! \wedge 1 \leq x\} a := 1 \{1 = a! \wedge a \leq x\}$  (ASI)
3.  $\{x > 0\} a := 1; y := 1 \{y = a! \wedge a \leq x\}$  (1, 2, SEC, CONS)

Hemos abreviado pasos. En el paso 3 se usa CONS considerando  $(x > 0) \rightarrow (1 = 1! \wedge 1 \leq x)$ .

## Prueba de (b)

4.  $\{y . a = a! \wedge a \leq x\} y := y . a \{y = a! \wedge a \leq x\}$  (ASI)
5.  $\{y . (a + 1) = (a + 1)! \wedge (a + 1) \leq x\} a := a + 1 \{y . a = a! \wedge a \leq x\}$  (ASI)
6.  $\{y = a! \wedge a \leq x \wedge a < x\} a := a + 1; y := y . a \{y = a! \wedge a \leq x\}$  (4, 5, SEC, CONS)
7.  $\{y = a! \wedge a \leq x\} \text{ while } a < x \text{ do } a := a + 1; y := y . a \text{ od } \{y = a! \wedge a \leq x \wedge \neg(a < x)\}$  (6, REP)
8.  $\{y = a! \wedge a \leq x\} \text{ while } a < x \text{ do } a := a + 1; y := y . a \text{ od } \{y = x!\}$  (7, CONS)

En el paso 6 se usa CONS considerando  $(y = a! \wedge a \leq x \wedge a < x) \rightarrow (y . (a + 1) = (a + 1)! \wedge (a + 1) \leq x)$ .

En el paso 8 se usa CONS considerando  $(y = a! \wedge a \leq x \wedge \neg(a < x)) \rightarrow (y = x!)$ .

## Prueba de (c)

9.  $\{x > 0\} S_{\text{fac}} \{y = x!\}$  (3, 8, SEC)

## Ejemplo 2. Prueba de terminación del programa del factorial

- Con el método H hemos probado:

$\{x > 0\} S_{\text{fac}} :: a := 1 ; y := 1 ; \text{while } a < x \text{ do } a := a + 1 ; y := y \cdot a \text{ od } \{y = x!\}$

utilizando el invariante  $p = (y = a! \wedge a \leq x)$

- Con el método H\* probaremos:

$\langle x > 0 \rangle S_{\text{fac}} :: a := 1 ; y := 1 ; \text{while } a < x \text{ do } a := a + 1 ; y := y \cdot a \text{ od } \langle \text{true} \rangle$

y de esta manera habremos probado:

$\langle x > 0 \rangle S_{\text{fac}} :: a := 1 ; y := 1 ; \text{while } a < x \text{ do } a := a + 1 ; y := y \cdot a \text{ od } \langle y = x! \rangle$

- Se propone:

Invariante  $p = (a \leq x)$

Al comienzo del while,  $a \leq x$ , y a la salida,  $a = x$ .

**Notar que este invariante es más simple que el utilizado antes.**

Variante  $t = x - a$

Al comienzo del while,  $t \geq 0$ .

A lo largo de sus iteraciones,  $t$  se decrementa en 1.

Y al final se cumple  $t = 0$ .

$\langle p \wedge B \rangle S \langle p \rangle , \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle , p \rightarrow t \geq 0$

$\langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle$

**Recordatorio de la regla REP\***

### Inicializaciones:

1.  $\langle x > 0 \rangle$   $a := 1$  ;  $y := 1$   $\langle a \leq x \rangle$  (ASI, SEC, CONS)

### Repetición:

Se tiene:  $p = (a \leq x)$ ,  $t = x - a$

Hay que probar por REP\* tres premisas:

Premisa 1:  $\langle p \wedge B \rangle S \langle p \rangle$

2.  $\langle a \leq x \wedge a < x \rangle$   $a := a + 1$  ;  $y := y \cdot a$   $\langle a \leq x \rangle$  (ASI, SEC, CONS)

Premisa 2:  $\langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle$

3.  $\langle a \leq x \wedge a < x \wedge x - a = Z \rangle$   $a := a + 1$  ;  $y := y \cdot a$   $\langle x - a < Z \rangle$  (ASI, SEC, CONS)

Premisa 3:  $p \rightarrow t \geq 0$

4.  $a \leq x \rightarrow x - a \geq 0$  (MAT)

Conclusión:  $\langle p \rangle$  while B do S od  $\langle p \wedge \neg B \rangle$

5.  $\langle a \leq x \rangle$  while  $a < x$  do  $a := a + 1$  ;  $y := y \cdot a$  od  $\langle a \leq x \wedge \neg(a < x) \rangle$  (2, 3, 4, REP\*)

### Programa completo:

6.  $\langle x > 0 \rangle$   $a := 1$  ;  $y := 1$  ; while  $a < x$  do  $a := a + 1$  ;  $y := y \cdot a$  od  $\langle \text{true} \rangle$  (1, 5, SEC, CONS)

En realidad, se probó  $\langle x > 0 \rangle S_{\text{fac}} \langle a = x \rangle$ . Pero la postcondición en este caso es irrelevante, el objetivo fue asegurar la terminación del while. La postcondición  $y = x!$  se alcanzó en la prueba de correctitud parcial.

$$\frac{\langle p \wedge B \rangle S \langle p \rangle, \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle, p \rightarrow t \geq 0}{\langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle}$$

**Recordatorio de la regla REP\***

### Programa

```
a := 1 ;  
y := 1 ;  
while a < x  
do  
  a := a + 1 ;  
  y := y . a  
od
```

### Proof outline

```
 $\langle x > 0 \rangle$   
a := 1 ; y := 1 ;  
 $\langle \text{inv: } a \leq x, \text{ var: } x - a \rangle$   
while a < x  
do  
  a := a + 1 ; y := y . a  
od  
 $\langle a \leq x \wedge \neg(a < x) \rangle$   
 $\langle \text{true} \rangle$ 
```

### Ejemplo 3. Prueba de correctitud parcial de un programa de división entera

Probaremos con H:  $\{x \geq 0 \wedge y > 0\} S_{\text{div}} \{x = y \cdot c + r \wedge r < y \wedge r \geq 0\}$ , con:

$S_{\text{div}} :: c := 0 ; r := x ; \text{while } r \geq y \text{ do } r := r - y ; c := c + 1 \text{ od}$

Proponemos como invariante del *while* la aserción:

$$p = (x = y \cdot c + r \wedge r \geq 0)$$

obtenida por una **generalización** de la postcondición del *while*.

Notar que cuando el programa termina se cumple  $r < y$ , y así de la conjunción de esta condición y el invariante se alcanza la postcondición buscada.

Podemos estructurar la prueba de la siguiente manera, que se desarrolla en el slide siguiente:

- a)  $\{x \geq 0 \wedge y > 0\} c := 0 ; r := x \{p\}$
- b)  $\{p\} \text{while } r \geq y \text{ do } r := r - y ; c := c + 1 \text{ od } \{p \wedge \neg(r \geq y)\}$
- c) Finalmente, aplicando SEC a (a) y (b), y como  $(p \wedge \neg(r \geq y)) \rightarrow x = y \cdot c + r \wedge r < y \wedge r \geq 0$ , por CONS se llega a:

$$\{x \geq 0 \wedge y > 0\} S_{\text{div}} \{x = y \cdot c + r \wedge r < y \wedge r \geq 0\}$$

$$\{p \wedge B\} S \{p\}$$

$$\{p\} \text{while } B \text{ do } S \text{ od } \{p \wedge \neg B\}$$

**Recordatorio de la regla REP**

Dada la precondition  $\{x \geq 0 \wedge y > 0\}$  y la postcondition  $\{x = y \cdot c + r \wedge r < y \wedge r \geq 0\}$ , usamos el invariante  $\{x = y \cdot c + r \wedge r \geq 0\}$ :

### Prueba de (a)

1.  $\{x = y \cdot c + x \wedge x \geq 0\} r := x \{x = y \cdot c + r \wedge r \geq 0\}$  (ASI)
2.  $\{x = y \cdot 0 + x \wedge x \geq 0\} c := 0 \{x = y \cdot c + x \wedge x \geq 0\}$  (ASI)
3.  $\{x = y \cdot 0 + x \wedge x \geq 0\} c := 0 ; r := x \{x = y \cdot c + r \wedge r \geq 0\}$  (1, 2, SEC)
4.  $(x \geq 0 \wedge y > 0) \rightarrow (x = y \cdot 0 + x \wedge x \geq 0)$  (MAT)
5.  $\{x \geq 0 \wedge y > 0\} c := 0 ; r := x \{x = y \cdot c + r \wedge r \geq 0\}$  (3, 4, CONS)

### Prueba de (b)

6.  $\{x = y \cdot (c + 1) + r \wedge r \geq 0\} c := c + 1 \{x = y \cdot c + r \wedge r \geq 0\}$  (ASI)
7.  $\{x = y \cdot (c + 1) + (r - y) \wedge r - y \geq 0\} r := r - y \{x = y \cdot (c + 1) + r \wedge r \geq 0\}$  (ASI)
8.  $\{x = y \cdot (c + 1) + (r - y) \wedge r - y \geq 0\} r := r - y ; c := c + 1 \{x = y \cdot c + r \wedge r \geq 0\}$  (6, 7, SEC)
9.  $(x = y \cdot c + r \wedge r \geq 0 \wedge r \geq y) \rightarrow (x = y \cdot (c + 1) + (r - y) \wedge r - y \geq 0)$  (MAT)
10.  $\{x = y \cdot c + r \wedge r \geq 0 \wedge r \geq y\} r := r - y ; c := c + 1 \{x = y \cdot c + r \wedge r \geq 0\}$  (8, 9, CONS)
11.  $\{x = y \cdot c + r \wedge r \geq 0\} \text{ while } r \geq y \text{ do } r := r - y ; c := c + 1 \text{ od } \{x = y \cdot c + r \wedge r \geq 0 \wedge \neg(r \geq y)\}$  (10, REP)

### Prueba de (c)

12.  $\{x \geq 0 \wedge y > 0\} c := 0 ; r := x ; \text{ while } r \geq y \text{ do } r := r - y ; c := c + 1 \text{ od } \{x = y \cdot c + r \wedge r \geq 0 \wedge \neg(r \geq y)\}$  (5, 11, SEC)
13.  $(x = y \cdot c + r \wedge r \geq 0 \wedge \neg(r \geq y)) \rightarrow (x = y \cdot c + r \wedge r < y \wedge r \geq 0)$  (MAT)
14.  $\{x \geq 0 \wedge y > 0\} c := 0 ; r := x ; \text{ while } r \geq y \text{ do } r := r - y ; c := c + 1 \text{ od } \{x = y \cdot c + r \wedge r < y \wedge r \geq 0\}$  (12, 13, CONS)

También se puede probar  $\{x \geq 0 \wedge y \geq 0\} S_{\text{div}} \{x = y \cdot c + r \wedge r < y \wedge r \geq 0\}$ , **permitiendo que el divisor y sea 0**. La prueba queda como **ejercicio**.

#### Ejemplo 4. Prueba de terminación del programa de división entera

Ya hemos probado mediante el método H:

$$\begin{array}{l} \{x \geq 0 \wedge y > 0\} \\ S_{\text{div}} :: c := 0 ; r := x ; \text{while } r \geq y \text{ do } r := r - y ; c := c + 1 \text{ od} \\ \{x = c.y + r \wedge 0 \leq r < y\} \end{array}$$

Notamos que también se puede probar con  $y \geq 0$ .

Ahora probaremos la terminación de  $S_{\text{div}}$  empleando REP\*. Naturalmente acá necesariamente la precondition debe incluir  $y > 0$ . Vamos a verificar:

$$\langle x \geq 0 \wedge y > 0 \rangle S_{\text{div}} \langle \text{true} \rangle$$

Se propone como **invariante** del while la aserción:

$$p = (x = c \cdot y + r \wedge r \geq 0 \wedge y > 0)$$

En este caso el invariante no es más simple que el de la prueba de correctitud parcial.

Y se propone como **variante** del while la función:

$$t = r$$

Claramente  $t$  siempre es positiva y se decrementa en cada iteración.

$$\langle p \wedge B \rangle S \langle p \rangle , \langle p \wedge B \wedge t = Z \rangle S \langle t < Z \rangle , p \rightarrow t \geq 0$$
$$\langle p \rangle \text{ while } B \text{ do } S \text{ od } \langle p \wedge \neg B \rangle$$

**Recordatorio de la regla REP\***



La prueba es la siguiente (para simplificar la escritura usamos directamente  $p$  en lugar de  $x = c.y + r \wedge r \geq 0 \wedge y > 0$ ):

**Prueba del fragmento inicial del programa:**

1.  $\langle x \geq 0 \wedge y > 0 \rangle c := 0 ; r := x \langle p \rangle$  (ASI, SEC, CONS)

**Prueba de las tres premisas de REP\*:**

2.  $\langle p \wedge r \geq y \rangle r := r - y ; c := c + 1 \langle p \rangle$  (ASI, SEC, CONS)

3.  $\langle p \wedge r \geq y \wedge r = Z \rangle r := r - y ; c := c + 1 \langle r < Z \rangle$  (ASI, SEC, CONS)

4.  $p \rightarrow r \geq 0$  (MAT)

**Se establece la conclusión de REP\* y la fórmula final pretendida:**

5.  $\langle p \rangle \text{ while } r \geq y \text{ do } r := r - y ; c := c + 1 \text{ od } \langle p \wedge \neg(r \geq y) \rangle$  (2, 3, 4, REP\*)

6.  $\langle x \geq 0 \wedge y > 0 \rangle S_{\text{div}} \langle \text{true} \rangle$  (1, 5, SEC, CONS)